Converge

THE CANONICAL BLUEPRINT · V6 — TWO PHASES, ONE BARRIER · THE CVG CLI · AGENT OPERATING PROTOCOL

The Converge System — Compile Intent, *Prove Every Step.*

The canonical blueprint for compiling a fuzzy brief — or a raw idea — into an autonomous, eval-gated system. **Nine passes, two phases, and one barrier** where the human hands off. Above the barrier, people make intent crystal-clear. Below it, the machine builds: sealed specs in, green-eval PRs out. A referee CLI — **cvg** — exposes every gate as a stable machine contract. v6 records both halves: what each pass means, and exactly how an agent discovers, runs, interprets, repairs, and hands off each one.

9	2	12	6	cvg
PASSES · 0–8 ONE PATH	PHASES · ONE BARRIER	CLAUDE CODE SKILLS	TASK SIZES XS → XXL	REFEREE CLI V0.16.0

DOCUMENT

Canonical process blueprint · v6 (agent-operable)

CONVERGED MEANS

an eval passed — not that you feel done

The invariant — every pass lowers altitude, binds an engine by flag, ends at a gate. Pass 8 is the exception: it does not lower, it closes. v6 supersedes v5: `Register` *is now* `Pass 6`*,* `Bind` *is* `Pass 7` *and absorbs the harness emit, the retired* `stack-to-harness` *skill is gone, and the Manager leaves the numbered chain entirely. Task-Spec* `3.6.0`*,* `cvg` `0.16.0`*. No pass is closed until its artifact, gate, proof, and lesson agree.*

Introduction

Why Converge exists

The old loop ran in your head. A requirement arrived fuzzy, and you held the definition of "done" as private judgment — refined by taste, defended in review, never written down. That loop does not scale to AI engines, because **an engine cannot execute a standard it cannot read**. The bottleneck was never typing speed; it was that intent lived in a human and had to be re-supervised at every step. The AI-native engineer inverts it: take a fuzzy requirement and lower it through a repeatable descent — each step more precise than the last — until "done" is a machine-checkable condition an engine can satisfy unsupervised.

The main idea — compile intent, don't write the system

Stop writing the system. **Compile it.** Converge treats building software as a compilation pipeline that lowers altitude in fixed stages: intent → structure → plan → tasks → code → loop. Each stage is a transformation with a typed output and a gate that must pass before the next stage runs — exactly how a compiler lowers source through intermediate representations down to machine code. You author the high-level intent and the checks; the pipeline emits the system. When no client brief exists, an optional on-ramp — **Pass 0 · Capture** — compiles the raw idea into the BRD that starts the descent.

The thesis — intent-driven development

The durable human job is making *intent crystal-clear*; the machine's job is figuring out *how*. Converge invests human effort where models stay weak — getting the intent right and verifying the result — and gets out of the way where they are strong — implementation. This is the horizon, and it holds for years: as more is automated, sending the *right intent* to the machine remains the one irreducible driver of building the right thing. So: reduce **procedural blockers** relentlessly; never reduce the **verification gate**. The isolated eval is not a blocker — it is the load-bearing property.

EVERY PASS

Lowers altitude

intent → structure → plan → tasks → code → loop. Each step closer to something a machine runs.

BINDS

An engine

the right tool for the altitude — a thinking session to shape, Claude Code to build, a different family to attack, a fleet to crank.

ENDS AT

A gate

converged means an eval passed — not that you feel done. The gate is the unit of trust.

> THE INVARIANT
>
> Every pass lowers altitude, binds an engine, ends at a gate. Pass 8 is the exception — it does not lower, it **closes**. You are converged when the eval passes, not when you feel done. And in the operating model, no pass is closed until its lesson exists.

Contents

The Whole Method, On One Line

`Capture⁰ → Intent → Structure → Decompose → Consensus (THE BARRIER) → Tasking → Register → Bind → Loop ↺`

A raw idea becomes a signed BRD, the BRD becomes a tech-spec, the tech-spec becomes ADRs, the ADRs become swimlane plans, the plans survive an adversarial review — **the owner signs off** — and from there the machine cuts tasks, registers a board, binds a runtime, and drives each issue to a green-eval PR. There is **one spine and no decision branch**: a six-tier engine sizes every unit from a one-liner to a whole backbone, so no separate spec-driven paradigm is needed. Each output is the next input.

#	PASS	SKILL	OUT · GATE
0	**Capture** (optional)	`idea-to-brd`	signed BRD — or a no-go record · `CHECK_BRD`
1	**Intent**	`brd-docs-to-tech-req`	tech-spec · answers the brief · `CHECK_TECH_SPEC`
2	**Structure**	`tech-req-to-adrs`	`docs/adrs/*` + `CONTEXT.md` · `CHECK_ADR`
3	**Decompose**	`reqs-to-swimlane-plans`	swimlane plans, legs · plan-altitude · `CHECK_PLAN`
4	**Consensus** THE BARRIER	`sketch-plans-adversarial-review`	plans attacked + sharpened · `CHECK_CONSENSUS` **+ the owner signs off**
5	**Tasking**	`task-spec` (six-tier)	XS/S/M/L leaves · XL/XXL nodes · HMAC `DELEGATE`
6	**Register** (opt-in)	`task-specs-to-issues`	1 spec = 1 issue · the board is state · `CHECK_REGISTER`
7	**Bind** (+ harness)	`task-to-runtime-contract`	profile + guards + `AGENTS.md` · `CHECK_RUNTIME_CONTRACT`
8	**The Loop**	`task-loop --issue N`	each task → green eval → PR closes the issue

> GUARDRAILS MAKE THE SEQUENCE CANONICAL
>
> Pass 0 never touches the repo — idea in, brief out; and Pass 1 must not consume an unsigned brief. Pass 2 writes ADRs only (no code). Pass 3 is plan-altitude only (no tasks, no SQL). Pass 4 always ends at the barrier — the sign-off is an output, never assumed earlier. Pass 8 does not lower — it closes. No pass is closed until its artifact, gate, proof, and lesson agree.

The spine, drawn — nine passes, two phases, one path

THE FULL DESCENT, AS A STRAIGHT LINE THROUGH ONE BARRIER

The descent is linear. An optional Capture on-ramp feeds it when no brief exists; four passes lower altitude; at Consensus the plans are hardened and **the owner signs off** — the barrier. From there the line runs on through Tasking, Register, Bind, and the Loop. No branch, no rejoin.

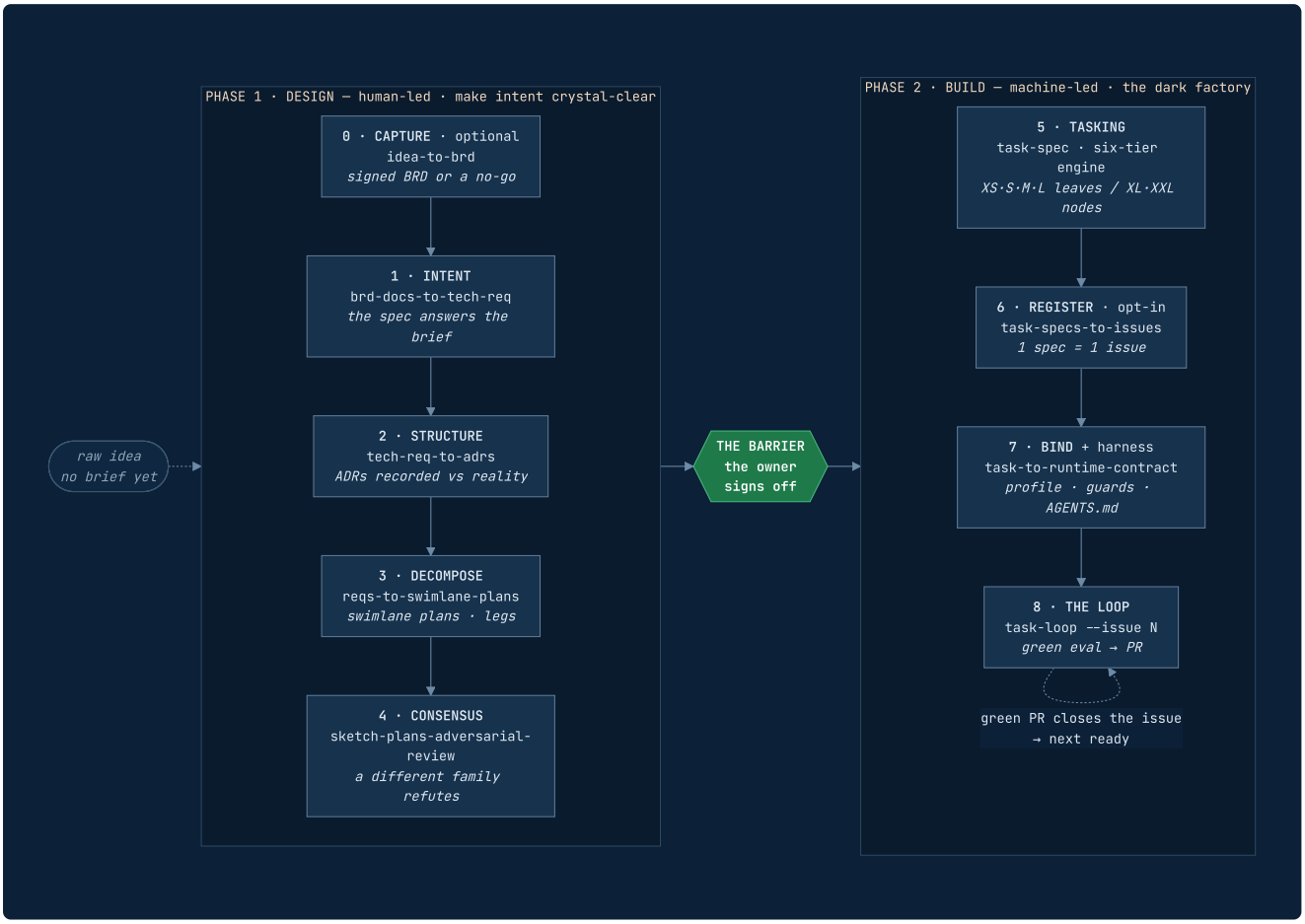


FIG 1 The nine-pass descent. Passes 0–4 are human-led design; passes 5–8 are machine-led build. Capture (0) and Register (6) are optional; everything else is on the required path. Pass 8 closes the loop rather than lowering altitude — a green PR closes its issue and unblocks the next.

READ THE DIAGRAM AS A CONTRACT

Each box is a transformation that must end in a stable token before the next box may run. The barrier is the only node that a machine cannot pass on its own — everything above it is judgment, everything below it is verification.

The Barrier — where the human hands off

THE SINGLE MOST IMPORTANT LINE IN THE METHOD · *this is the L4 → L5 boundary*

Pass 4 ends by doing one thing: **the owner signs off.** Every pass before it is human-led — you are making intent crystal-clear. Every pass after it is machine-led — the sealed spec is the only instruction, and the eval is the only judge of done. That single line is the whole shape of the method.

	ABOVE THE BARRIER (0–4)	BELOW THE BARRIER (5–8)
who	human-led design	machine-led build
the work	make intent crystal-clear	turn intent into green-eval PRs
the artifact	BRD → spec → ADRs → plans → sign-off	sealed task-specs → issues → contracts → merged PRs
the human's job	own the intent + the verification	walk out at the gate

Why here, and not earlier?

Because this is the first moment the plan has survived *a different model trying to break it.* Signing off before that is signing off on an unexamined plan — and the machine would then faithfully build the flaw, quickly and at scale. **The adversary is what makes the sign-off worth something.**

THE HALF NO SCRIPT CAN CHECK

`check-consensus-gate.sh` proves the machine-checkable half: a different-family adversary attacked the live plans (provenance stamp + input hashes, not a grep of a word), every objection is FIXED or ACCEPTED-with-owner, and the plans have not drifted since. It **cannot** prove the sign-off. An agent that reaches a green consensus gate with no owner signature must stop and say so — not proceed to Pass 5.

WHERE THIS SITS IN THE FIELD

Dan Shapiro's *Five Levels: from Spicy Autocomplete to the Dark Factory* (Jan 2026) tops out at **L5 — the dark factory**: specs in, software out, no human reading the diff. Converge's **Phase 1 is the L4 work** (write and argue the spec, craft the skills); **Phase 2 is L5**. The barrier at Consensus *is* the L4 → L5 line.

0 No brief yet? Compile the idea into one.

PASS 0 OF 8 · OPTIONAL ON-RAMP · ALTITUDE: IDEA — *receive & interrogate the raw idea*

ENGINE	THINKING SESSION	SKILL	IDEA-TO-BRD V0.6.0	OUT · SIGNED BRD OR NO-GO

Not every engagement starts with a client brief. Pass 0 takes a raw idea with no document behind it and compiles it into a BRD written *in the owner's voice* — or kills it cheaply. It scope-checks that this is one idea, grills the owner in **frontier rounds** (each round asks every question whose prerequisites are settled, numbered, each with a recommended default; one reply answers the whole round), then drafts the brief and self-reviews it against the gate.

Two honest exits: a signed BRD that Pass 1 can consume unchanged, or a **no-go record** parked with the reasoning intact. A gate that can only pass or "not yet" pressures the interview to manufacture numbers; the no-go exit removes that pressure.

SCOPE-CHECK

One idea

is this one idea, or four? Explore the environment for facts first — facts never enter a round, and never block one.

GRILL

Frontier rounds

seven branches in leverage order — pain → do-nothing test → goal → scope boundary → success → constraints → pre-mortem.

DRAFT

BRD + self-review

`docs/brd-<slug>.md` in the owner's voice, then audited against the gate before sign-off.

GATE

The pain carries a **provenance-tagged number** — `(measured) / (estimated) / (guessed)` — with at least one KPI in the owner's terms; scope in/out are non-empty; every open question has a named owner; the do-nothing test and pre-mortem ran. The gate speaks in tokens — `CHECK_BRD=PASS|FAIL|DRAFT_OK|NOGO_OK` — and **canonical** must sit on the Sign-off verdict line. Pass 1 must not consume an unsigned brief, and a no-go here is the **cheapest kill in the whole descent**.

The frontier-rounds grill adapts Matt Pocock's `batch-grill-me`; Pass 0 adds the fixed branch map, the no-go exit, provenance tags, and the gate.

1 The client hands you the problem. You produce the spec.

PASS 1 OF 8 · ALTITUDE: INTENT — *receive & comprehend the brief*

ENGINE	THINKING SESSION · --ENGINE COWORK	SKILL	BRD-DOCS-TO-TECH-REQ V0.6.0	OUT · TECH-SPEC

Read the BRD like a senior engineer, not a stenographer. Find the real problem underneath the ask, name who actually hurts when it is unsolved, and pin the numbers that define success. **Ambiguity killed here is the cheapest kill in the whole spine**, and the most expensive to leave alone — it compounds through every pass downstream.

The interrogation runs in frontier rounds; facts never enter a round. Unanswered questions ride twice, then become owned open questions. What survives becomes a typed **gap register** (`GAP-001 · scope | definition | number | data · blocker | minor`): a blocker left open fails the gate. Prior art is checked at the problem level only — never the codebase, which is Pass 2's altitude.

UNDERSTAND

Read

the one-paragraph "solved" statement comes first: the real problem underneath, who hurts, the numbers that define success.

INTERROGATE

Grill

the 2–3 questions whose answers most change how this gets built; ambiguity here compounds downstream.

CRYSTALLIZE

Spec

replay the "Locked:" recap, then resolve into one technical answer — falsifiable requirements, metrics traced to the BRD's KPIs.

GATE

You can restate the client's problem **in one breath** and show the tech-spec answers it — scope, data, and success metric named, every requirement falsifiable, every blocker gap closed. `--draft` validates structure but never authorizes; the canonical run passes only a spec whose Sign-off verdict line reads **canonical**. `CHECK_TECH_SPEC=PASS` is the only verdict that hands to Pass 2.

THE MOST COMMON PASS 1 FAILURE

No premature technology. No schema, no data store, no framework names. Describe *what* the system must do and *how well*. Brief-in, spec-out — never blur them: the BRD is the client's, in the client's words; the tech-spec is yours. The stack is decided at Pass 3, not here.

2 Greenfield or brownfield — and write the ADRs.

PASS 2 OF 8 · ALTITUDE: SYSTEM — *ground the spec, record the decisions*

ENGINE	CLAUDE CODE · ON THE REPO	SKILL	TECH-REQ-TO-ADRS V0.4.0	OUT · DOCS/ADRS/*.MD + CONTEXT.MD

Before any plan, **name the ground**. Brownfield is a system you did not write: open it end to end, and check the spec against the real schemas, sources, and jobs you find. Greenfield is a system that does not yet exist: confirm there is nothing to inherit, then ground the spec in the source systems and constraints it must honour. Either way you leave Pass 2 knowing the terrain — what is actually there, and what the work must respect.

Not every fact earns an ADR. The **worthiness test** keeps the record honest: a decision qualifies only if it is *hard to reverse*, *stood on downstream*, and *could have been otherwise*. Shared vocabulary goes to `docs/CONTEXT.md` — the glossary every later pass speaks, pinned as terms crystallize.

NAME THE GROUND

Field

brownfield (a system you didn't write) vs greenfield (nothing yet) — name it explicitly; the rest follows.

GROUND

Check

open the system end to end; reconcile the spec against real schemas, sources, and jobs.

RECORD

ADRs

write `docs/adrs/*.md` — grounding decisions that pass the worthiness test. Facts and constraints, never how to build.

GATE

The system is understood and its grounding decisions are recorded durably, so Pass 3 can read them without re-deriving the terrain. `CHECK_ADR=OK` . An ADR records what *is* true — schemas, joins, constraints — never how to build.

THE SINGLE FAILURE MODE OF THIS PASS

The moment an ADR says **"build X"**, you have drifted into planning and must split it: the fact stays here, the design moves to Pass 3. The `--check` gate detects build-verbs for exactly this reason. *An ADR says what is true;* `CONTEXT.md` *says what things mean.*

3 Split the work along its natural seams.

PASS 3 OF 8 · ALTITUDE: PLAN — *decomposition · break it into swimlanes*

ENGINE	CLAUDE CODE · SAME SESSION AS PASS 2	SKILL	REQS-TO-SWIMLANE-PLANS V0.7.0	IN · READS THE ADRS

Take the system understood in Pass 2 — same session, ADRs in hand — and cut it where it is *already jointed*: at a feature edge or a component edge, not an arbitrary slice. Each seam becomes one swimlane; each swimlane gets its own focus-oriented sketch plan. **The right number of lanes is the number of genuine seams** — no more, no fewer.

The decomposition chain is **seam → swimlane → leg → task-spec**. A *leg* is a lane's named stretch: one responsibility, one proving test, independently finishable in order. Legs are where Pass 5 will later cut 1:N task-specs — but the leg itself never carries a runnable eval.

DECOMPOSE

Seams

cut where the system is already jointed. Prefer the **highest seam, and the fewest** — every seam is an interface someone must freeze, attack, and honour.

SWIMLANE

One plan each

every seam becomes one lane carrying exactly one sketch plan — one lane, one plan, one focus.

GROUNDED

Inherits ADRs

each lane stands on `docs/adrs/`. A plan that contradicts an ADR, or silently re-settles it, fails the gate.

GATE

One sketch plan per genuine seam, each listing legs, dependencies, build order, and the exact upstream interface it consumes — inheriting the ADR decisions. Legs are present, named, and **contiguous** (`leg-01..leg-0N`). `CHECK_PLAN=OK|FAIL|EMPTY|USAGE_ERROR` . Plan-altitude only.

THE ONE GUARDRAIL THAT DEFINES THE PASS

The moment a plan holds a query body, a handler body, or an atomic task with an eval, it has left Pass 3 and skipped the adversarial review meant to harden the plan *first*. A green `--check` proves the legs exist, are named, and are ordered — it does **not** prove they are well-cut. Whether a leg is genuinely independently-finishable is Pass 4's judgment, not a grep's.

4 Bring a different engine to refute the plans — then sign off.

PASS 4 OF 8 · ALTITUDE: PLAN (HARDENED) · **THE BARRIER**

ENGINE	CROSS-FAMILY ADVERSARY (CODEX / KIMI)	SKILL	SKETCH-PLANS-ADVERSARIAL-REVIEW V0.6.0	CLI · CVG REVIEW

A single model won't refute itself hard enough. So Pass 4 brings a **different-family** engine to attack the plans, one lane and one leg at a time, as a skeptic who defaults to "refuted." It grounds each plan against the tech-spec and the ADRs — hunting contradictions, gaps, and silent assumptions — and every objection is either **FIXED** in a plan or **ACCEPTED** with a named owner. Nothing is silently dropped. Cross-model disagreement is not noise here; it is the entire point.

When argument can't settle an objection, settle it with a **throwaway prototype** — the smallest disposable artifact that answers the question, run against real data. The result decides FIX or ACCEPT; the prototype is *evidence, not deliverable*. Then the pass does the one thing that defines it: **the owner signs off.**

ATTACK

Refute

a different family, one plan at a time, as a skeptic who didn't write it. Default to refuted — silence is not passing.

GROUND

Drift?

check each plan against the tech-spec and the ADRs — contradictions, gaps, silent assumptions, disagreeing numbers.

SHARPEN

Fix / Own

every objection FIXED in a plan or ACCEPTED with a named owner — nothing silently dropped.

GATE — TWO HALVES

Machine (falsifiable, not honour-system): the gate reads a **stamped objection log**, never plan prose. It passes only when a **different-family** adversary attacked — proven by a provenance stamp plus input hashes computed by the referee, not a grep of a word — every objection is FIX-or-ACCEPT-with-owner, and the plans have **not drifted** since the review (re-hashed). `cvg review --check` → `CHECK_CONSENSUS=OK` .

Human: the owner signs off. This is the barrier, and no script can check it.

THE COMPATIBILITY TOKEN

The objection-log schema still *requires* a `fork` object, and the gate still requires a `FORK: B` line atop every swimlane PRD. This is a **frozen compatibility field, not a decision** — the method has one path. Agents must never present it to the owner as a route to pick.

One spine — no fork, no route decision.

TASKING IS THE ARCHITECTURE · RECURSION ABSORBS SCALE

Converge does not fork at Consensus. A different-family adversary attacks the plans, every objection is settled, the owner signs — and the hardened plan tree goes directly to Tasking. Frontier engines execute well-scoped atoms reliably, and a tree of verified atoms composes back up more safely than one monolith. A single **six-tier Task-Spec engine** therefore carries the whole range — from a one-line change to a complete backbone — without a second specification paradigm.

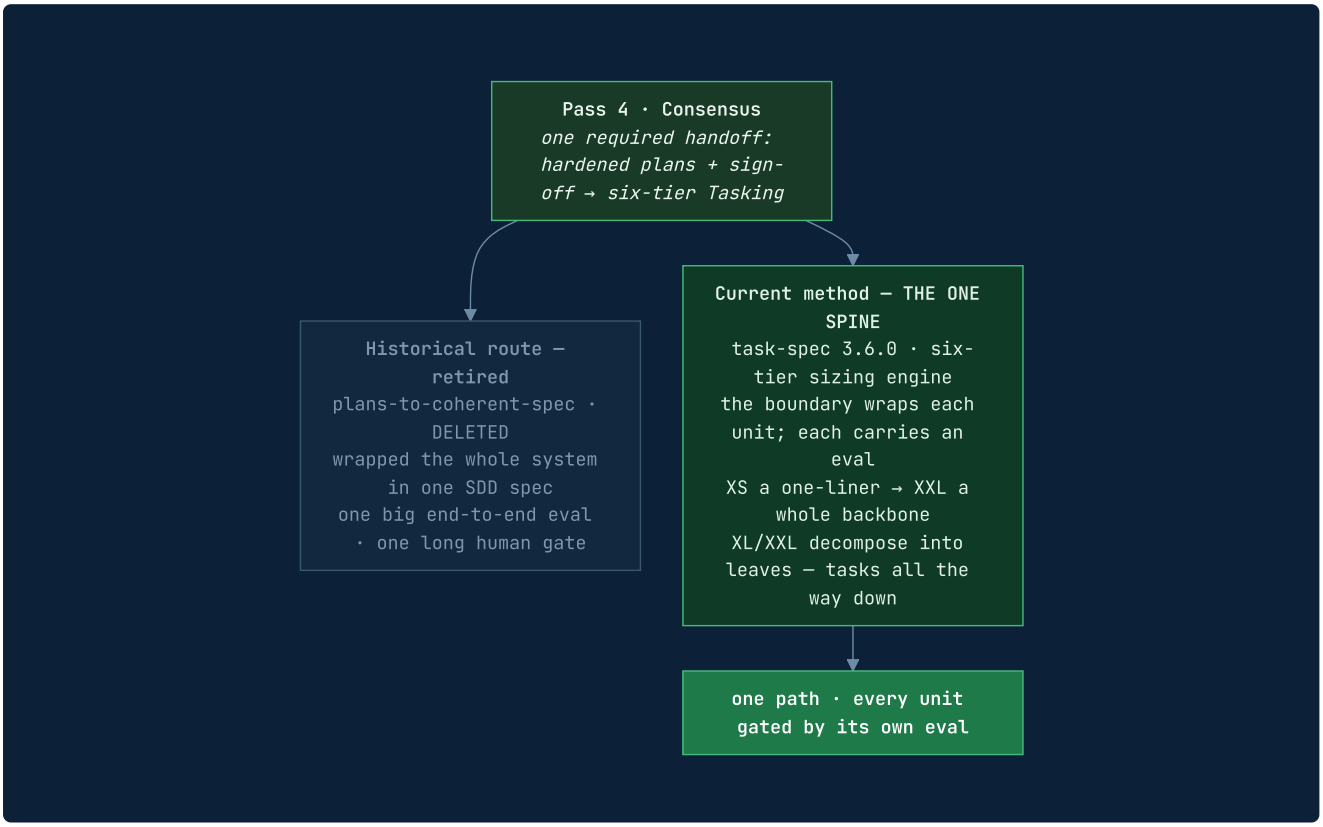


FIG 3 There is no architectural choice after Consensus. The schema still stamps `FORK: B` for backward compatibility with the consensus gate; agents treat it as a required compatibility value, never as a prompt to choose a branch.

THE SIX-TIER ENGINE ABSORBS THE RANGE

Leaves run · nodes decompose

Leaves (XS/S/M/L) are runnable atoms — each fits one fresh context window and verifies as one PR or test-suite; a gate-checked **write-surface budget** catches a mis-sized unit. **Nodes** (XL/XXL) are not runnable — they MUST declare `children:` and expand into leaves. A worker dispatches the children, never the node.

WHY ONE PATH IS SAFER

Recursion, not escalation

Big work used to route **out** to SDD. Now it decomposes **in**: the same task-spec format, self-similar at every scale, so the seam contracts are written once and per-unit tasks can't drift.

TASKS ALL THE WAY DOWN — AND BACK UP

The dark factory needs one unit model at every scale. A subjective slice becomes a human-checkpoint eval; a coupled slice becomes an **L leaf**; a whole backbone becomes an **XXL node** that decomposes to a tree of leaves. There is no route out: descend by decomposition, run only leaves, then compose verified results back up.

5 The six-tier engine — size every unit, from XS to XXL.

PASS 5 · THE SIZING MODEL · TASK-SPEC V3.6 — *one engine, a one-liner to a whole backbone*

Every unit gets a t-shirt size — sized the agent-native way: not by human hours, but by **context, verification, and blast radius. Leaves** (XS/S/M/L) are runnable atoms that fit one fresh context window. **Nodes** (XL/XXL) are decomposition directives — not runnable.

SIZE	KIND	WRITE BUDGET · BACKEND	EXAMPLE
XS	leaf	≤ 1 path	a config key; bump a dependency
S	leaf	≤ 2 paths	a `/health` endpoint + its test
M	leaf	≤ 3 paths	a dbt model + schema + test
L	leaf · ceiling	≤ 5 paths · long-horizon backend	a service slice as ONE coherent goal
XL	node · decomposes	children ≥ 2 (owns none)	a whole swimlane → its leg leaves
XXL	node · decomposes	children ≥ 3 (owns none)	a whole backbone → swimlane nodes

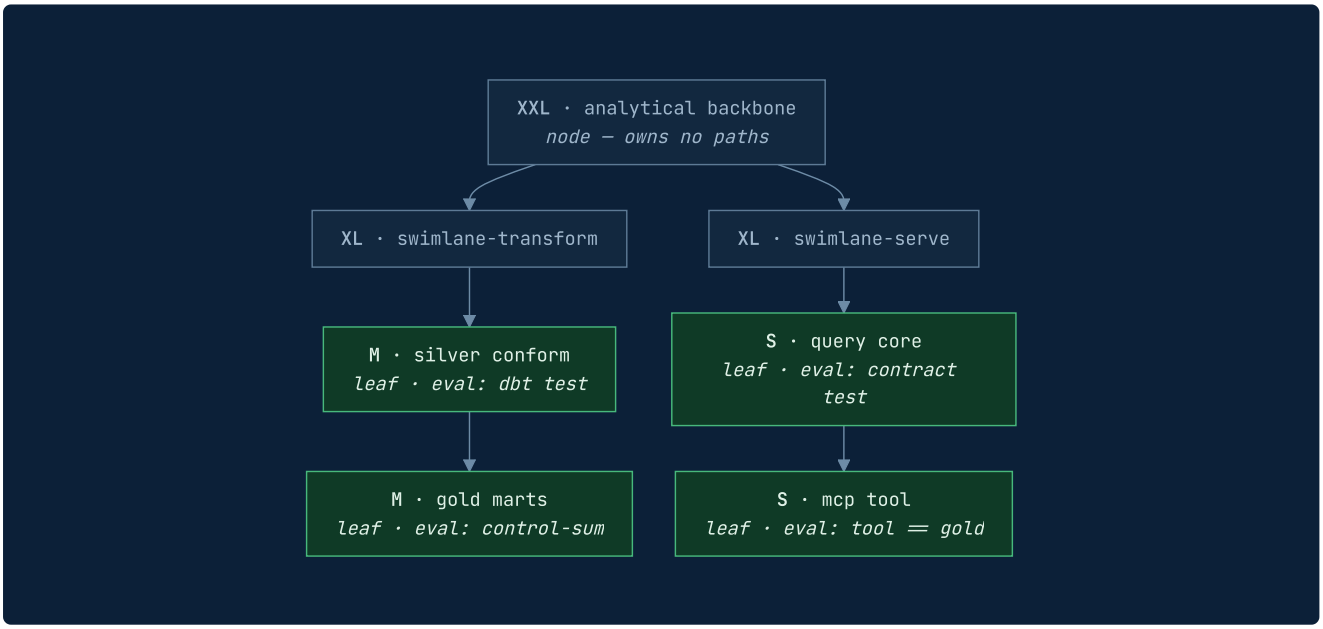


FIG 4 The recursion — this is the whole idea. A worker runs the **LEAVES** and composes their verified results back up the tree. A budget that gets breached means the decomposition was too coarse — the signal to split further. Big work decomposes *in*; it never routes *out*.

GATE — CVG TASKS VALIDATE

Sizing is **enforced, not advisory**: a leaf over its write-surface budget WARNs "mis-sized — split or reclassify UP"; an XL/XXL node with too few children FAILs "must decompose — no route out"; L requires a **long-horizon** `execution_backend` and one coherent done-condition. As of v3.6 that is a fleet-configurable set (`glm claude codex kimi` by default), not one hard-coded vendor — the format stays vendor-neutral and self-verifying.

5 Tasking, in practice — a backbone into eval-backed units.

PASS 5 OF 8 · ALTITUDE: ATOMIC UNIT — *the plans decompose into task-specs*

SKILL | TASK-SPEC V3.6.0 | CLI · CVG TASKS VALIDATE | GATE | DOD

Consensus hands off here, always. The hardened swimlane plans decompose into **eval-backed task-specs** — the trust boundary wraps each unit, and verification is per-unit. Trust is earned one unit at a time: each eval that goes green is a gate you no longer hold by hand. This is the path to the Dark Factory — many small, automatable gates a fleet can close on a schedule. On the proving ground, the whole analytics backbone is already tasked: **9 legs → 9 task-specs**, wired into a dependency DAG, every one `validate` ok · `gate` DELEGATE · `dod` COMPLETE.

Each unit is a **vertical slice** — a tracer bullet, not a horizontal layer — sized by the six-tier engine; behaviors (`B-1` , `B-2` ...) trace to evals and back; and the seam contracts hardened at Pass 4 ride in as guardrails, so a per-unit task can't drift from the consensus.

SIZE

Six tiers

XS/S/M/L leaves run; XL/XXL nodes decompose. The write-surface budget catches a mis-sized unit at the gate.

BIND EVAL

Done = ?

a runnable eval defines done — the test passes, the query returns N, the probe reaches a served answer.

TRACE

DoD matrix

`cvg tasks dod` renders every behavior → its verifying eval → result; an untraced behavior FAILS the DoD.

GATE — CVG TASKS GATE (SAFE-TO-DELEGATE)

Every task carries a runnable eval — **no eval, not a task yet**. The delegation gate runs two stages: structural + sizing validation, then a broken-eval guard — and **refuses to delegate an XL/XXL node** (a worker dispatches its children). Sign-off is a key-optional **HMAC seal**, **tamper-evident**: it proves the evals were not weakened after sign-off, so the sealed spec becomes the only trusted instruction source. Self-contained and vendor-portable: Claude, Codex, Kimi, or manual.

6 Each Task-Spec becomes one tracked issue.

PASS 6 OF 8 · *opt-in, like Capture* · THE TRACKER AS STATE

Register projects every `signed_off: true` Task-Spec onto exactly one tracker issue — a disposable **state shadow** of the repo-owned work. The tracker is a pluggable backend behind **six verbs**: `preflight · upsert · link · list-ready · list-issues · write-result`. Upsert is idempotent by the spec ID; each `depends_on` edge becomes one `blocked-by` relation; the task's Exit Check travels into the issue as its close condition. After a successful upsert, Register writes only a receipt — `tracker_ref` — back into frontmatter. **The issue-side marker remains the idempotency key**, so CI can use `--no-stamp-refs` without losing convergence.

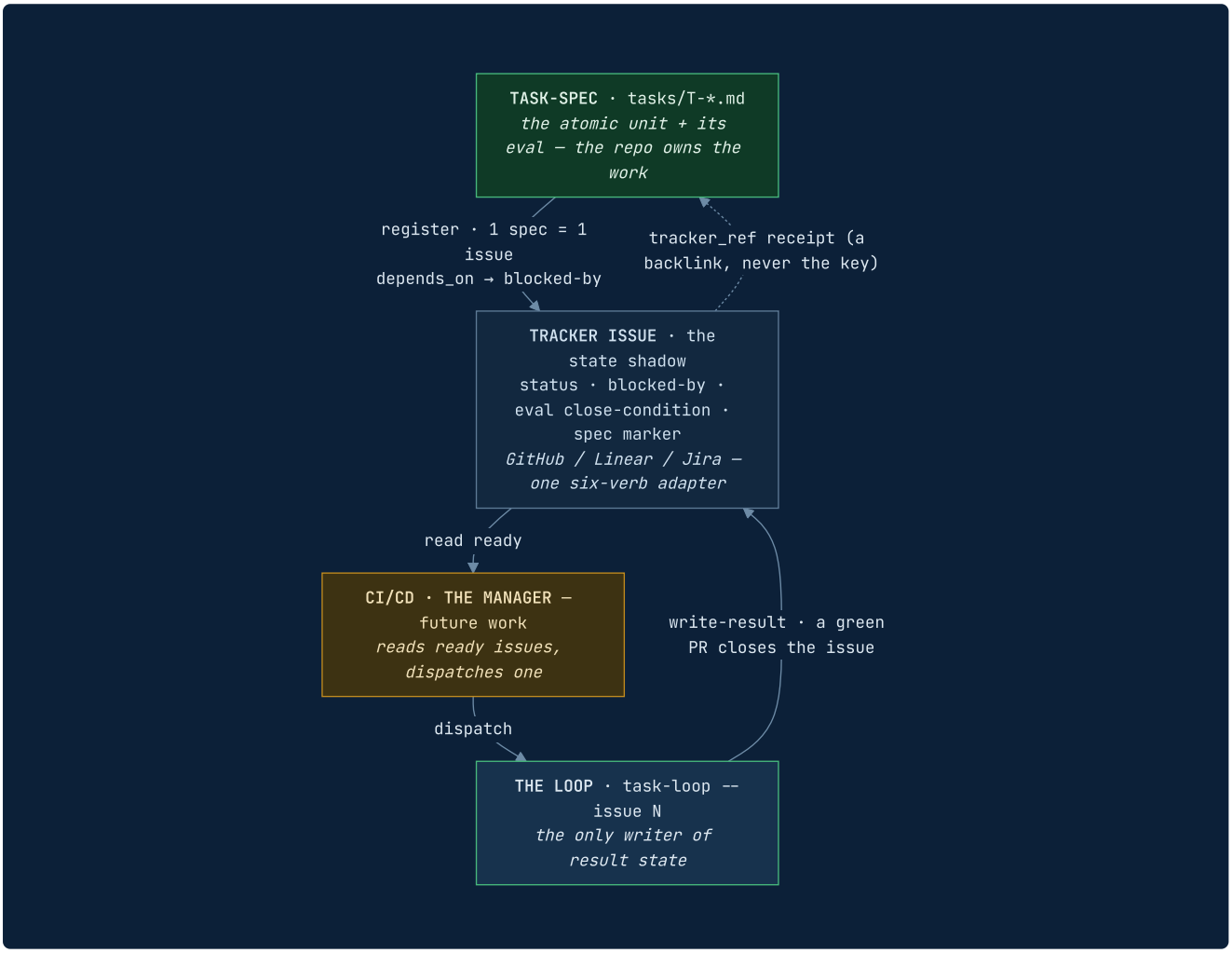


FIG 5 The repo owns the work; the tracker carries its dispatchable shadow. Only the loop writes result state, and only a green eval closes an issue — which is why the board never lies.

A REAL PARITY GATE

It can fail

`cvg register --check` uses the read-only `list-issues` verb to prove `count(issues) == count(signed-off specs)`, every edge is one link, no orphans, no cycles, no un-gated leak.

THE SOURCE AND THE SHADOW

Skip it freely

Stay repo-local when you want the specs to travel with the code and need no external system. Add Register when you want a board the loops read across sessions and machines. The tracker is replaceable shared memory; the repo files remain authoritative and can rebuild it.

7 Bind one task to its runtime — and emit the harness.

PASS 7 OF 8 · ALTITUDE: RUNTIME CONTRACT — *authority in motion*

Pass 7 makes two moves. **(7a) Contract:** verify one signed runnable Task-Spec, bind its exact hash to the *smallest* evidence slice it needs, pick a topology, and emit portable path guards. **(7b) Harness:** auto-detect the available engines and emit the multi-engine context glue — `AGENTS.md` plus Claude / Codex / cloud adapters — so the *same* sealed task runs anywhere.

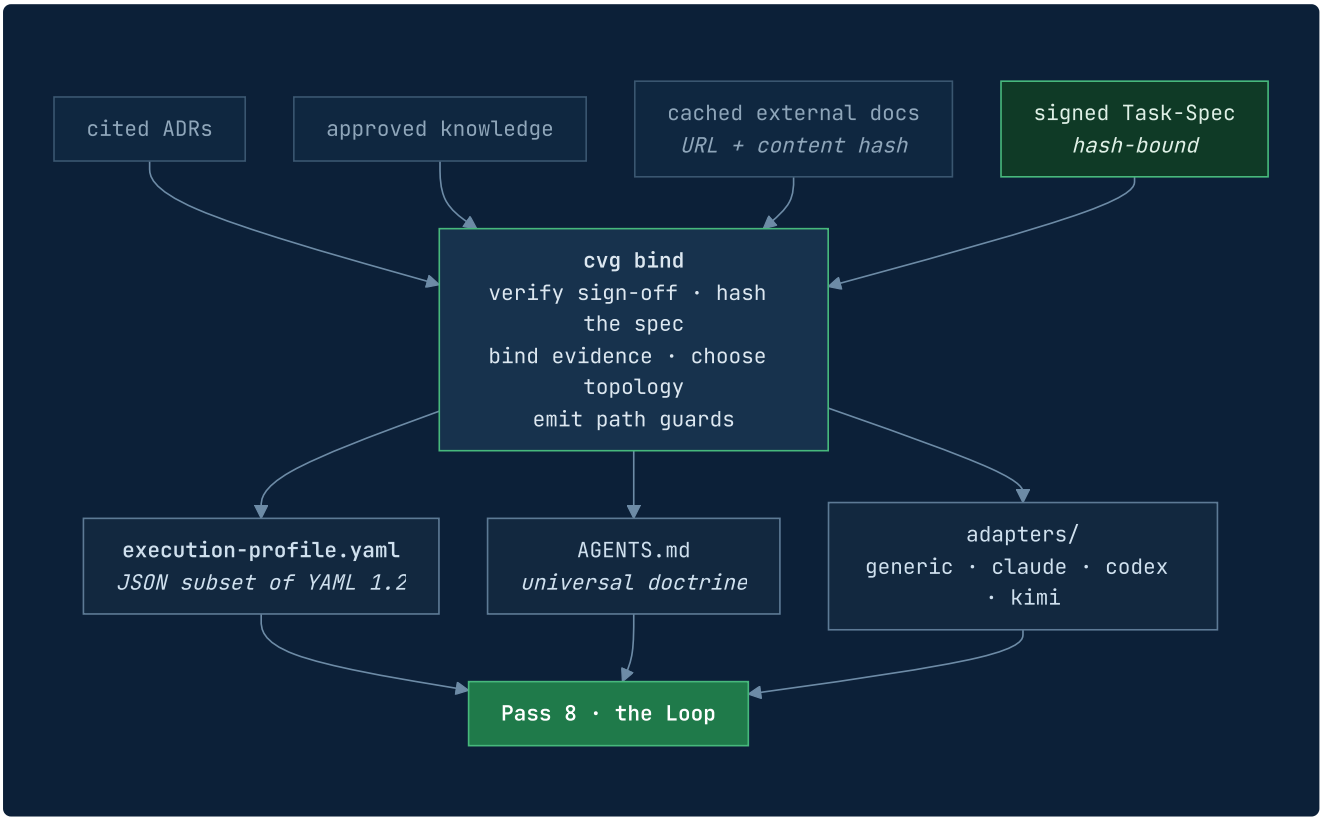


FIG 6 Bind is task-scoped and hash-bound. External docs must have a local cached copy; the profile records URL and content hash so offline, version-pinned execution stays possible.

GATE — CHECK_RUNTIME_CONTRACT=PASS

The Task-Spec is a signed runnable **leaf** and its current hash matches; required ADRs, knowledge, and cached docs exist and match their recorded hashes; a non-`single` topology carries a substantive *static* justification; parallel write ownership is explicit and disjoint; guards and all declared adapter manifests exist; no unresolved placeholders remain.

THE CAPABILITY ENVELOPE — AUTHORITY THAT CLOSES

Bind's central claim is not "the agent was told what it may touch." It is that **authority is granted against one signed revision, scoped to that revision's own paths, and revoked the moment the task settles.** The envelope is keyed to an **epoch** — `<task-id>@<spec-sha12>` — so changing a byte of the spec voids every grant derived from it. This answers *lingering authority*: coding-agent CLIs grant permissions for a *session*, and a session outlives a task. Each adapter declares, per capability, whether the runtime **prevents** the violation, only **detects** it, or **cannot honor** it — and a required control the runtime cannot enforce **fails the gate closed**, unless waived in the open.

The boundary Pass 7 defends — the harness orchestrates; the documentation teaches. External docs are cached and hashed, approved knowledge referenced, never copied. Topology defaults to `single` and escalates only for a reason visible before runtime.

8 Build the execution loop. Schedule the Manager later.

PASS 8 OF 8 · ALTITUDE: RUNTIME — *loop engineering*

Loop Engineering is the discipline of designing the loops that prompt agents, rather than prompting agents. The unit of work is no longer the message but the cycle. One primitive recurs everywhere — take an action, read the feedback the environment returns, decide the next move, repeat until the **goal condition** holds. Convergence is defined by the goal, never by attempt-count: a loop that stops because it tried three times has *not* converged; a loop that stops because the eval is green has.

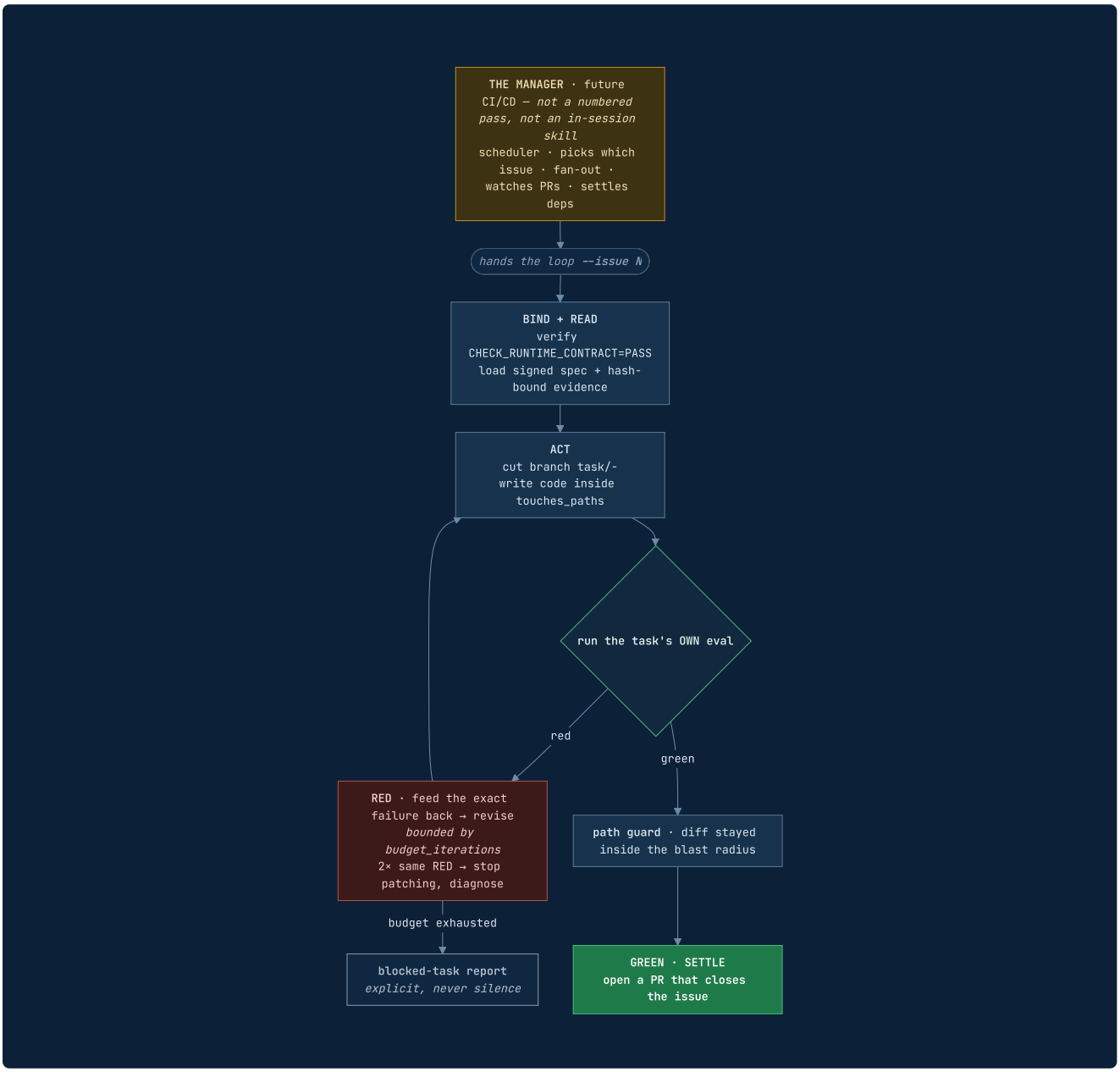


FIG 7 The execution loop takes ONE issue, writes code, runs the task's eval, feeds RED back until GREEN, checks the path guard, then opens a PR. The Manager that picks which issue and fans out is a future CI/CD concern — GitHub Actions as scheduler, the PR as state settlement, branch protection as the gate.

GATE

The task's own eval is green and the runtime path policy holds — **none by hand, none by attempt-count.** Green-eval-closes-the-issue is the dark factory, one issue at a time.

The architecture, whole.

SKILLS AUTHOR · THE REFEREE ADJUDICATES · ENGINES EXECUTE · TRACKERS SHADOW

Four layers, and the discipline is to keep them separate. **Skills** transform artifacts. The **cvg CLI** adjudicates — it holds zero model credentials and makes no LLM calls. **Engines** are commodity slots bound by flag. **Trackers** carry a disposable shadow of repo-owned state.

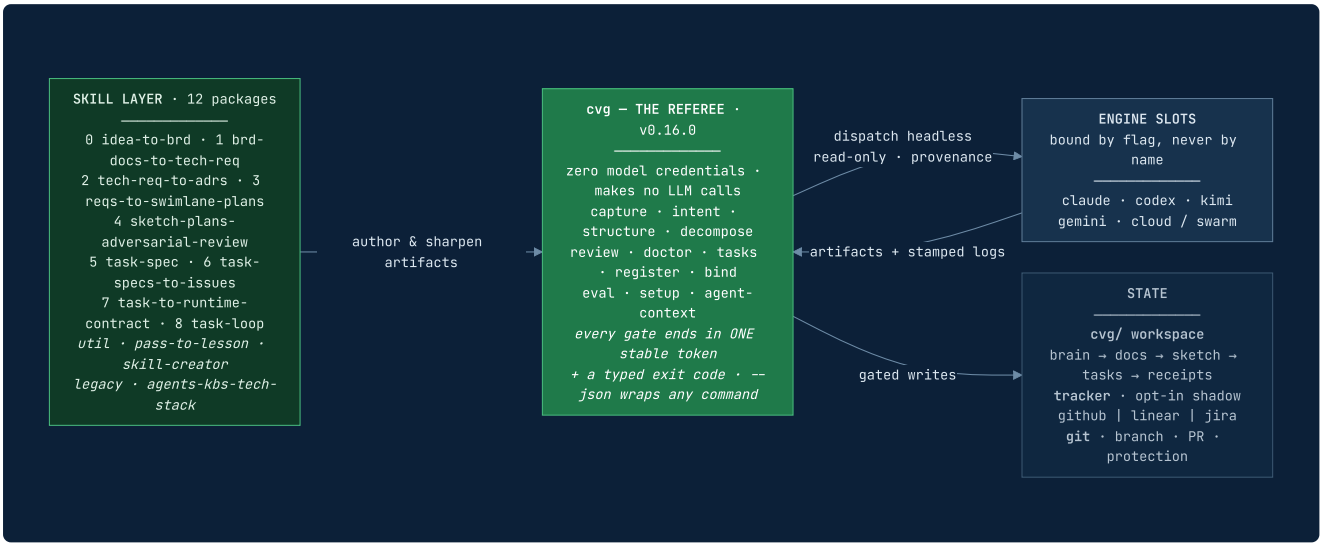


FIG 8 The referee never authors and the skills never adjudicate. That separation is what makes the evidence trustworthy — an agent that asks the referee to be the player has removed it.

THE TWO CONTRACTS

Task-Spec = authorization and done. Because the *eval* — not the agent — defines done, any coding agent is a swappable commodity slot. **Runtime contract = authority in motion.** It binds the exact signed revision to evidence, topology, permissions, and concrete guards. Keep them separate. *Run on commodity models; produce defensible output.*

The operating model — five beats per pass.

HOW THE METHOD BUILDS ITSELF · TRACK R CURRENT · TRACK M PARKED

Converge is built with its own method. **Track R** validates the method one pass at a time, and each pass is a complete vertical slice of five beats — **in order, never bundled**. **Track M** — the execution machine, fixture-tested — waits and resumes later.

BEAT	WHAT HAPPENS · WHERE IT LANDS
1 · UNDERSTAND	read the pass's skill end to end, compare against the field, deliver a hardening list — analysis only, no edits
2 · RUN	execute the pass **for real** on the proving ground; the owner judges every output
3 · CANONIZE	iterate skill + artifact until the owner calls it canonical — blueprints are **canonical v0**; a later pass may send a retro-edit back
4 · IMPLEMENT	the `cvg` subcommand for this pass — frame → dispatch → collect → gate, thin by design
5 · PROVE	re-run the pass through the CLI on the same input; artifacts and gate verdicts must match the golden manual run — **nothing to trust, only to diff**

THE TEACH BEAT — INSIDE THE C-BEAT

Closing CANONIZE includes the TEACH beat: run `pass-to-lesson` on the just-canonized artifacts — every component taught (what it is, why it's shaped this way, the decision it encodes, what breaks downstream without it), `check-lesson.sh` green, the lesson committed under `docs/lessons/`. **A pass is not closed until its lesson exists.** Rule 0 is the stance: go slow and teach — the owner's understanding is a deliverable equal to working code. The companion explains decisions, never reopens them.

WHAT AUTONOMY ACTUALLY COSTS

Autonomy is the reward for understanding, not a substitute for it. Converge earns the right to run dark at Pass 8 by spending human judgment at Passes 1–4 — the swimlanes, the ADRs, the adversary. A running loop with no upstream comprehension builds the wrong thing flawlessly. **The factory must run dark and be aimed correctly; these passes are what aim it.**

The knowledge home — the cvg/ workspace.

FIVE FOLDERS, ONE LIFECYCLE EACH

Every consuming project gets one Converge home — `cvg/`, named for the CLI — with `INDEX.md` as the front door. Artifacts flow one way and never backward: **brain feeds → docs agree → sketch explores → tasks execute → receipts prove.**

FOLDER	KIND	LIFECYCLE
`brain/`	user-generated inputs	raw, append-only, never gated
`docs/`	consensus artifacts	gated, permanent, never deleted — BRD · tech-spec · `adrs/` + `CONTEXT.md` · `lessons/`
`sketch/`	drafts	transient — sharpened at Pass 4, decomposed into tasks at Pass 5
`tasks/`	sealed execution units	HMAC-stamped, tracked
`receipts/`	evidence — gate verdicts, pass receipts	write-once

ONE HOME

Per project

the workspace is named for the method's CLI, not the tool of the moment; dot-space (`.cvg/`) stays reserved for machine config.

ONE DIRECTION

Never backward

inputs feed, agreements gate, drafts expire, units seal, evidence freezes — nothing flows upstream.

ONE TRUTH

The repo

second-brain tools pull from the repo, never the reverse — the repo stays the single source of truth.

THE CONVENTION

The repo stays the single source of truth; second-brain tools and trackers are **projections**. `.cvg/` records non-secret choices only — credentials live in the environment or a secrets manager, never in project state. Conversation memory is orientation, never proof.

The cvg CLI — a referee, never a player.

AGENT-NATIVE · V0.16.0 · ZERO MODEL CREDENTIALS

Every pass is gated by one CLI — `cvg` — wrapping the proven skill scripts. It holds **zero model credentials** and makes no LLM calls: it frames prompts, dispatches engine CLIs headlessly, and gates their output. Wrapped gates are **byte-exact pass-throughs**; bash-3.2-safe, stdlib-only, dep-free in CI. Every gate ends in ONE stable, greppable token, and exit codes are contracts — a harness never parses prose.

```
# the whole nine-pass chain — one name, one token each
cvg capture    → CHECK_BRD=PASS          cvg decompose → CHECK_PLAN=OK
cvg intent     → CHECK_TECH_SPEC=PASS    cvg review --adversary codex,kimi → REVIEW=OK
cvg structure  → CHECK_ADR=OK            cvg review --check → CHECK_CONSENSUS=OK
cvg tasks validate | gate | accept | dod  cvg doctor → DOCTOR=OK
cvg register [--check] → CHECK_REGISTER=OK
cvg bind [--check] --task <spec> → CHECK_RUNTIME_CONTRACT=PASS

# agent-native — wrap ANY command in one uniform envelope
$ cvg --json capture
{ "ok":true, "command":"capture", "token":"CHECK_BRD=PASS",
  "exit_code":0, "changed":false, "error":null,
  "meta":{ "schema_version":"1.0", "cvg_version":"0.16.0" } }

cvg agent-context  → command tree + exit-code taxonomy (retryable), as JSON
cvg setup tracker linear → discover team · record choice, never key
cvg <mutation> --dry-run  → reports intent, changes nothing (changed:false)
```

AGENT-NATIVE

--json · agent-context · dod

a uniform `--json` envelope on every command, an exit-code taxonomy with `retryable` / `side_effects`, a self-describing manifest, `--dry-run` on mutations, and the DoD/traceability view — all shipped.

DOGFOODING

Nothing to trust, only to diff

Every subcommand reproduces the canonical manual run — artifacts and gate verdicts diffed against the golden reference — before it ships. Gates ship as versioned exit contracts whose regression suites prove the *negatives* fail: a fixture that wrongly PASSes is caught, because evals must discriminate.

THE RESULT

A referee you can point an autonomous engine at — it never bluffs a pass.

PART II · OPERATIONAL SPECIFICATION

The Agent Operating *cvg.*

The first half explains the method. This half explains the machine behavior: how an autonomous agent discovers the available surface, creates the right artifact at the right altitude, invokes the referee, interprets a stable verdict, repairs only the owning stage, and advances without inventing state.

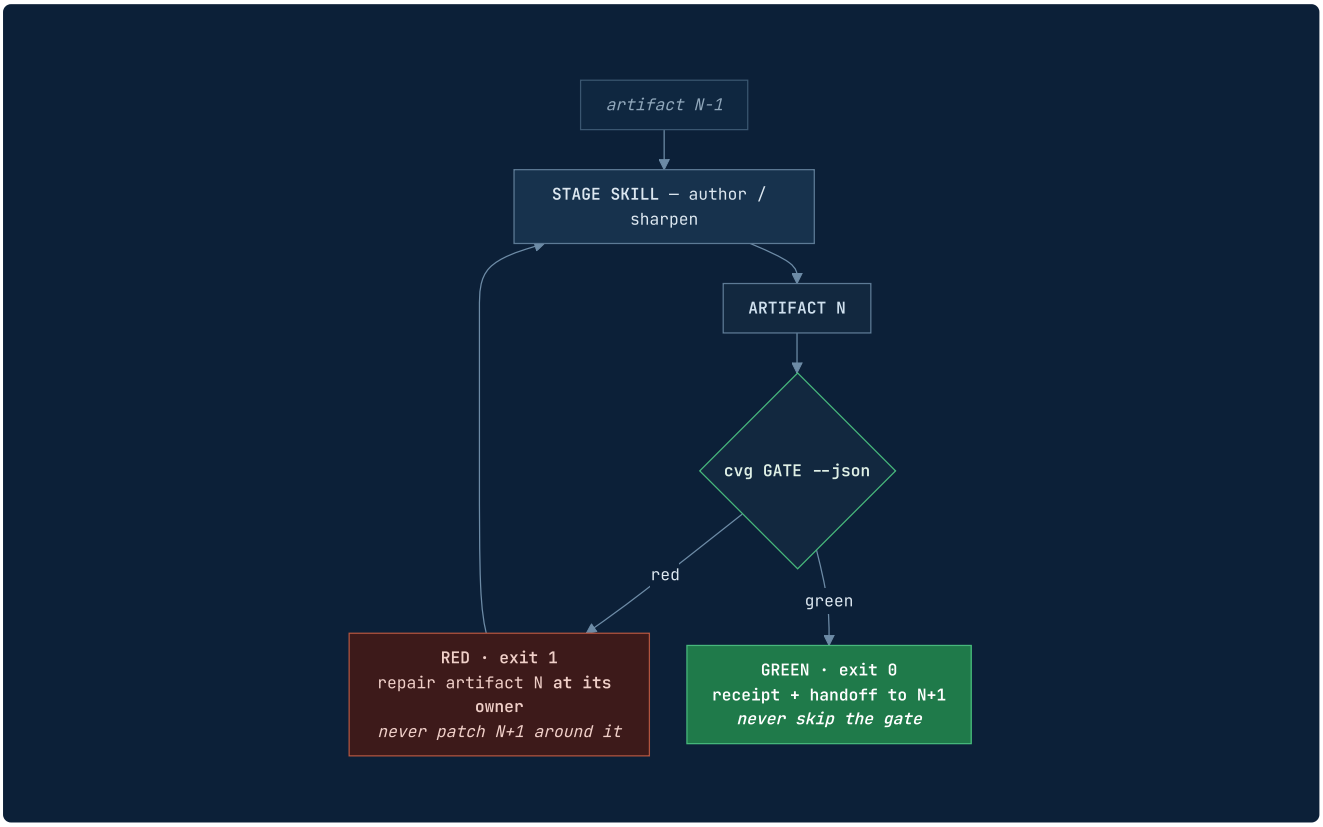
```
DISCOVER agent-context → AUTHOR with the stage skill → GATE through cvg → INTERPRET token + exit
→ REPAIR OR HAND OFF
```

One protocol, repeated at every altitude. The CLI is the referee; the skills and engine sessions are the players. The agent never mistakes validation for authorship.

The universal agent protocol.

THE SAME CONTROL LOOP AT EVERY PASS · ONLY THE ARTIFACT AND GATE CHANGE

An agent does not "run Converge" as one giant prompt. It advances a typed state machine one gate at a time. Each pass owns one transformation; `cvg` publishes the evidence contract that says whether the transformation is allowed to hand off.



#	STEP	AGENT RULE
1	**Discover**	Call `cvg agent-context` once. Learn command names, tokens, mutability, retryability, and the CLI version without scraping help prose.
2	**Locate state**	Read `cvg/INDEX.md`, then the artifact owned by the current pass. **Never infer progress** from a branch name, tracker column, or conversation memory.
3	**Author**	Use the stage skill to create or sharpen the artifact. The CLI validates; it does not replace the authoring skill or the stakeholder decision.
4	**Preview**	For a mutation, prefer `--dry-run` or the stage's read-only check. Confirm targets, blast radius, credentials boundary, and expected receipt before changing state.
5	**Gate**	Invoke `cvg … --json`. Trust the exit code plus stable token — do not infer success from a pleasant sentence, an engine exit 0, or an artifact merely existing.
6	**Settle**	GREEN → persist proof and hand the typed output to the next pass. RED → repair the artifact **in the pass that owns it**, rerun the same gate, and never patch downstream around an upstream gap.

THE MOST IMPORTANT DISTINCTION

A skill **transforms**; `cvg` **adjudicates**. `cvg capture` does not interview the owner, and `cvg intent` does not write the tech-spec. An agent that asks the referee to be the player has removed the very separation that makes the evidence trustworthy.

Machine contracts — how the agent decides.

TOKENS · JSON ENVELOPE · EXIT TAXONOMY · MUTATION SEMANTICS

An autonomous caller needs a smaller, stricter surface than a human. Every `cvg` command therefore ends in a stable token and preserves a typed exit code. `--json` wraps the same command without changing the underlying verdict or the byte-parity default path.

EXIT	MEANING	AGENT RESPONSE
0	**OK**	Persist receipt and advance. For a mutation, confirm `changed` matches expectation.
1	**GATE_FAIL**	Artifact is red. Repair at the current pass; a blind retry is not progress.
2	**USAGE_ERROR**	Bad flag, ambiguous discovery, or nothing to gate. Fix invocation or identify the artifact.
20	**SKIP**	Engine unavailable. Select an allowed ready engine or stop with an explicit coverage gap.
21	**TIMEOUT**	Retryable dispatch failure. Scope smaller or retry within policy; **no fabricated artifact**.
22	**ENGINE_ERROR**	Provider output could not become a valid judgment. Preserve raw output and diagnose adapter/format.

READ-ONLY BY DEFAULT

Gate without changing

`capture`, `intent`, `structure`, `decompose`, checks, lint, ready, eval, manifest.

MUTATIONS ARE EXPLICIT

Preview, then act

adversary dispatch, acceptance stamps, transitions, setup choice, registration, bind. Use `--dry-run` where supported.

RECEIPTS, NOT STORIES

Persist the verdict

store the token, exit, version, artifact hash/path, and external ref. Human prose is context, never the gate.

THE RULE THAT PREVENTS FABRICATED PROGRESS

Exits **20 / 21 / 22** are *infrastructure* conditions and are retryable; exit **1** is an *artifact* gate failure and must be repaired, not blindly retried. An agent that cannot tell these apart will eventually invent an artifact to satisfy a gate — which is precisely the failure the whole evidence design exists to prevent.

Evidence ledger — what exists, what is proved.

A BLUEPRINT BECOMES MISLEADING WHEN IT TREATS SHIPPED AND PROVED AS THE SAME STATE

Code-complete, fixture-proven, live-connected, and proving-ground-closed are four different states. v6 keeps them explicit.

SURFACE	STATE	EVIDENCE BOUNDARY
Passes 0–4	CLOSED	Canonical proving-ground artifacts, direct-gate parity, and a real cross-family Consensus with provenance re-hash.
Tasking · Pass 5	CLOSED	Nine backbone legs are nine specs; all `validate` ok, `gate` DELEGATE, `dod` COMPLETE.
Register · Pass 6	LIVE-PROVEN	Six-verb adapters; 120/120 offline suite via the `fake` backend; live Linear registration exercised (CVG-21…29) with a real 1:1 parity gate.
Bind · Pass 7	IMPLEMENTED	Binder, readiness gate, path guard, adapters, receipts exist; 19/19 regression checks green. Full proving-ground closure still ahead.
Loop · Pass 8	SINGLE ISSUE	`task-loop` executes one named issue; `cvg eval` runs evals. Manager, fan-out, CI board automation remain planned.
cvg core	SHIPPED	0.16.0: pass gates, agent-context, JSON, dry-run, DoD, setup, register, bind.
Task-Spec	3.6.0	Six-tier engine, dual gates, HMAC envelope, L0–L2 conformance suite.

IMPLEMENTED MEANS

It exists on disk

the command or skill exists and its documented contract can be inspected. It may also have fixture proof. **This does not automatically close its method round.**

CLOSED MEANS

It ran, for real

the real pass ran on the proving ground, the artifact was canonized, the CLI reproduced the golden gate, the proof is durable, and the owner can be taught what closed.

THE NAMED NEXT STEP

The field's litmus test for a *real* dark factory is three load-bearing properties: agents work from **written specs** (Converge ✅), an **isolated evaluator grades against holdout the coder cannot see** (Converge ⚠️ — this is the promoted backlog item), and the **deploy path is unchanged** (Converge ✅). Stating the gap is the point.

Why this is defensible

TWO MOATS · POSITIONING · THE HONEST BOUNDARY

The two moats

OPTIONALITY

Task-Spec

the eval — not the agent — defines done, so you swap Codex / Kimi / Claude freely; the coding agent is a commodity slot.

QUALITY

Harness

the control-plane files that make any commodity agent succeed; not downloadable, built over thousands of hours.

TOGETHER

Defensible

run on commodity models, produce defensible output. Two moats — optionality *and* quality — not one.

Positioning vs the field

The canonical 2026 spec-driven pipeline is **Specify → Plan → Tasks → Implement**, punctuated by human checkpoints — Spec Kit, Kiro, OpenSpec, and BMAD all share that shape, and their gates and loops keep growing. Converge's claim is not a feature checklist but a **difference in contract**: the full front-of-funnel intent descent (0–4) where most tools start at "you already have a spec"; the adversarial **Consensus** pass where a *different-family* model attacks each plan default-to-refuted; an **explicit barrier** instead of implicit checkpoints; **HMAC-sealed specs** as the only trusted instruction source (injection defense); the **one-spine model** that decomposes big work in place rather than routing to a second paradigm; and the closed **Loop** of Pass 8, where every unit is born with a runnable eval and only a green eval closes an issue.

THE HONEST BOUNDARY

The Loop converges to **green evals, not to correct outcomes**. A passing spec test guarantees only that the code matches the spec — if the spec is wrong, the code faithfully implements the wrong thing. The Dark Factory automates the *build*, not the *judgment of whether the spec was right*. The defense is structural and partial, not total: Phase 1 exists entirely to make the intent right, Pass 1's gate forces the spec to answer the brief, and Pass 4's adversary attacks it before any code is written. **Stating this plainly is the point — the intellectual honesty is the moat's foundation, not a footnote.**

The runtime, proven

WHAT THE SKILLS ALREADY ENFORCE — NOT A PROMISE, A MECHANISM

The gates above are not honour-system prompts. The task-spec skill ships the runtime that makes "done = proved" a mechanism a human cannot fake and an agent cannot shortcut. Five enforcement layers matter most, and all five already exist on disk.

LAYER	WHAT IT ENFORCES · WHERE IT LIVES
Atomic state machine	A locked, portable transition protocol — advisory lock, read current status, validate the transition against a real state machine, write via tmp-file + atomic move. Two agents cannot both claim one ready task.
Tool-captured evidence	The acceptance gate **re-runs the evals itself** — it never trusts the executor's word. A non-zero exit is rejected automatically.
Anti-reward-hacking	Blast-radius check (changed files must be inside `touches_paths`), HMAC contract integrity (evals were not weakened after sign-off), and **gold-sanity** — the eval must FAIL on the unpatched baseline.
Rebuildable ledger	The spec frontmatter is truth; the metrics ledger is append-only; the state index is rebuildable. The board is a projection of one source of truth — it cannot silently diverge.
Teaching gate	Every artifact taught, every decision naming a rejected alternative, every term of art defined, 3–5 check-yourself questions. **A pass is not closed until its lesson exists.**

One method, every engineer

THE ROLE LIVES IN THE EVAL, NOT THE LOOP

The loop never inspects *what* a task is — only whether its eval is green. All role-specificity is pushed down into the per-task eval, so the role does not change the machine; it only changes the assertion the machine waits on.

ROLE	WHAT ITS TASK'S EVAL IS
Data Engineer	a dbt test + a control-sum query against gold
Software Engineer	a unit / integration test that passes
DevOps	`terraform plan` clean + a smoke test
AI Engineer	a retrieval-precision or eval-set threshold met

The method, end to end on one feature

POSTGRES → DUCKDB → DBT → MCP · THE WORKED EXAMPLE

One feature carries the whole framework: **"daily revenue per product category, served via MCP so a non-engineer can ask for it."** Every pass runs on the real repo.

PASS	WHAT HAPPENS ON THE REAL REPO
0 · Capture	Raw idea → `docs/brd-analytical-backbone.md` in the owner's voice, signed. *Gate: check-brd green.*
1 · Intent	BRD → tech-spec: one gold model + one MCP tool over raw orders + payments + products. *Gate: the spec answers the brief.*
2 · Structure	Brownfield. Open `src/db/01_schema.sql`; write ADRs `0001-join-on-order-id`, `0002-utc-date-grain`, `0003-revenue-is-paid-only`.
3 · Decompose	Two lanes, plan-altitude only: transform (`silver → gold_daily_revenue`) and serve (`mcp tool query_daily_revenue`).
4 · Consensus THE BARRIER	`cvg review --adversary codex` attacks: *"midnight off-by-one (no TZ)? refunds counted?"* FIX: pin UTC (ADR-0002). ACCEPT: refunds out of v1, owner named. **The owner signs off — barrier crossed.**
5 · Tasking	`build_gold_revenue` (eval: dbt test + control-sum) and `build_mcp_revenue` (eval: tool result == gold query). Each born with its eval, HMAC-sealed.
6 · Register	`--tracker linear` : `build_gold_revenue → ISSUE-41 [ready]` ; `build_mcp_revenue → ISSUE-42 [blocked-by 41]` .
7 · Bind	`cvg bind --task build_gold_revenue.md` : bind the signed hash to ADRs + cached dbt docs + single topology + path guards; emit `AGENTS.md` → `CHECK_RUNTIME_CONTRACT=PASS` .
8 · The Loop	`task-loop --issue 41` : contract PASS → dbt test RED (null categories) → add `COALESCE` → GREEN + path policy PASS → PR. 41 done, 42 unblocks.

THE DARK FACTORY, ON THIS REPO

A non-engineer asks the MCP *"revenue by category yesterday?"* — and it answers. Built brief → deploy, every step eval-verified, **the human walked out at the barrier**.

COLOPHON · PROVENANCE

You are converged when the eval passes — *not when you feel done.*

This blueprint was reconciled against the live repository: the twelve `SKILL.md` contracts, `bin/cvg` and its surface ledger, the Task-Spec changelog, the six-verb adapter contract, `PLAN.md`, and the `tests/uc-analytics` proving ground. Where a surface is implemented but not yet proven end-to-end, the evidence ledger says so plainly rather than rounding it up.

VERSION

Method v6 · supersedes v5

COMPONENTS

12 skills · cvg 0.16.0 · Task-Spec 3.6.0

LICENCE

MIT · built by Luan Moreno

What changed from v5 — Register entered the numbered chain as Pass 6, moving Bind to Pass 7 and folding the harness emit into it; the `stack-to-harness` skill was deleted and the skill count fell from 13 to 12; the Manager left the numbered chain to become a future CI/CD concern; and the barrier at Consensus was promoted from an implicit handoff to the organising idea of the whole method.